

Homework 2

1 Colorful Path

1.1 Pseudocode

Algorithm 1: ColorfulPath(G)

```
/* G: input adjacency list as a nested hash table */
/* P: output list of tuples of four nodes that are "colorful" */
/* For this algorithm, 1-based indexing was used. */
/* Task: Design an algorithm that takes as input a graph  $G$  and returns all node
   quadruples that form a "colorful" path (colorful: sequence of red -> blue ->
   green) */
1  $P \leftarrow$  empty list[tuple(length of 4)] /* initialize the output list P of tuples of four
   nodes */
2 for  $u$  in  $G$  do
3   for  $v$  in  $G[u]$  do
4     /* first 2 nested for loops:  $n + m$  operations */
4      $colorful \leftarrow$  empty tuple of length of 4. /* create an empty tuple to record the
       possible permutation */
5     if  $G[u][v] == 'r'$  AND  $G[v][u] == 'r'$  then
6       /* checking if the edge between  $u$  and  $v$  is red (undirected; we have to
         check both) */
6        $colorful[1] \leftarrow G[u][v]$  /* push  $u$ th node into the tuple: initial node  $u$  */
7        $colorful[2] \leftarrow G[v][u]$  /* put  $v$ th node into the tuple:  $u$  connected with  $v$  */
8       /* third nested for loop:  $D$  operations */
9       for  $r$  in  $G[v]$  do
10        /* we have to iterate through  $v$ th dictionary or edges of node  $v$  to
          look for 'blue' edge */
10        if  $G[v][r] == 'b'$  AND  $G[r][v] == 'b'$  then
11          /* checking if the edge between  $v$  and  $r$  is blue (undirected; we
            have to check both) */
11           $colorful[3] \leftarrow G[v][r]$  /* if it's red->blue, put  $r$ th node into the
            tuple:  $v$  connected with  $r$  */
12          /* fourth nested for loop:  $D$  operations */
12          for  $b$  in  $G[r]$  do
13            /* we have to iterate through  $r$ th dictionary or edges of node  $r$ 
              to look for 'green' edge */
13            if  $G[r][b] == 'g'$  AND  $G[b][r] == 'g'$  then
14              /* checking if the edge between  $r$  and  $b$  is green
                (undirected; we have to check both) */
14               $colorful[4] \leftarrow G[r][b]$  /* if it's red->blue->green, put  $b$ th node
                into the tuple:  $r$  connected with  $b$  */
15               $P.append(colorful)$  /* now that we have a "colorful" tuple,
                append the tuple to list  $P$ ; this operation is repeated
                until the whole loops terminate */
16 return  $P$  /* after appending all possible tuples, return the list of tuples. */
```

1.2 Runtime Analysis

1. $O(1)$: initialize the empty list of tuples of four nodes (P) to return.
2. $O(n + m)$: first two nested for loops that iterates through the end of the input nested hash table G . Since the length of each Node is different, the upper bound of these two nested loops would be $n + m$.
 - 2-1. $O(1)$: initialize the empty tuple of length of 4 for "colorful" tuple of 4 to filter only the "colorful" ones. Put first 2 nodes u and v into **colorful** tuple. At this point, we can find red.
 - 2-2. $O(D)$: loop through the edges of Node v to look for proceeding color. Put third node r into **colorful** tuple. At this point, we can find red→blue.
 - 2-2-1. $O(D)$: loop through the edges of Node r to look for proceeding color. Put fourth node b into **colorful** tuple. At this point, we can find red→blue→green.
 - 2-2-2. $O(1)$: Since we found 1 "colorful" path, append the tuple to the list.

To sum all the steps, at maximum, there can be $n + m(D \cdot D)$ operations for this algorithm. Therefore, the running time of 'ColorfulPath(G)' is $O(n + mD^2)$.

2 Part 1: Asymptotics

2.1 $f(n) = n^{0.9}$ and $g(n) = \sqrt{2n}$

Answer: $f(n) = O(g(n))$ If we rewrite $g(n)$,

$$\begin{aligned} & \sqrt{2n} \\ &= (2n)^{0.5} \\ &= 2^{0.5} \times n^{0.5} \end{aligned}$$

Let $c = 2^{0.5}$ and $n_0 = 0$ and $n_1 = 1$

Case 1: $n_0 = 1$

$$f(n) = 1^{0.9} = 1 < g(n) = 2^{0.5} \times 1^{0.5} = \sqrt{2}$$

Case 2: $n + 1 = 2$

$$f(n) = 2^{0.9} < g(n) = 2^{0.5} \times 2^{0.5} = 2$$

So, there exist constants $c = 2^{0.5} > 0$ and $n_0 = 1$ such that $\forall n \geq n_0, f(n) \leq c \cdot g(n)$. Therefore, $f(n) = O(g(n))$.

2.2 $f(n) = 2^n + n^2$ and $g(n) = n^6$

Answer: $g(n) = O(f(n))$

According to the growth-rate rule of thumb,

$$n^c \ll a^n \text{ for any constant } c > 0, a > 1$$

$$\text{Let } c = 6, a = 2$$

$$n^c = n^6 = g(n) \ll a^n = 2^n$$

For the remainder of f(n), n^2 can be ignored.

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \frac{2^n + n^2}{n^6} > 0 \text{ such that } f(n) = \Omega(g(n)).$$

Therefore, $f(n) = \Omega(g(n))$ such that $g(n) = O(f(n))$.

2.3 $f(n) = \log_2(n), g(n) = \log_2(n^2 + n)$

Answer: $f(n) = O(g(n))$

$$g(n) = \log_2(n^2 + n)$$

$$= \log_2(n(n + 1))$$

$$= (\text{Use log product rule}) \log_2(n) + \log_2(n + 1)$$

$$\text{So, } f(n) \leq c \cdot g(n) \text{ for any constant } c > 0, n_0$$

Therefore, $f(n) = O(g(n))$.

2.4 $f(n) = n^2 \log_2(n), g(n) = n^2 \cdot \sqrt{n}$

Answer: $g(n) = O(f(n))$

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \frac{n^2 \log_2(n)}{n^2 \cdot \sqrt{n}}$$

$$= \frac{\log_2(n)}{\sqrt{n}} = \frac{\log_2(n)}{n^{0.5}}$$

According to growth-rate rule of thumb:

$$\log(n) \ll n^c = \log_2(n) \ll n^{0.5}$$

$$\text{So, } \frac{\log_2(n)}{n^{0.5}} > 0$$

$$\text{So, } f(n) = \Omega(g(n))$$

Therefore, $g(n) = O(f(n))$.

3 Part 2: Runtime Analysis

3.1 Basic Merging

1. $O(1)$ to initialize the empty list C .
2. ‘while’ loop until either list A or B is empty to remove the smaller of the first elements of A and B and append it to C . The worst case would be to iterate through the end of BOTH A and B lists ($O(ab)$)
3. there are three cases of append task:

1. $a > b$
 1. In this case, the append job would take $O(a - b)$ times.
2. $b > a$
 1. In this case, the append job would take $O(b - a)$ times.
3. $a = b$
 1. In this case, we do not have append task.

Therefore, $O(1) + O(ab) +$ and either $O(a - b)$ or $O(b - a) = O(ab)$.

3.2 Merging k Sorted Lists

$O(n)$ for the first initialization of R by copying L_1 . For the worst case, the upper bound of the ‘for’ loop operation would be $(k - 1)n^2$ as the $\text{Merge}(R, L_i)$ Operation is $O(n^2)$ and we repeat for $k - 1$ times. We subtract the number of ‘Merge’ operation by 1 as we already did once for the initialization. Therefore, it is $O(n) + O((k - 1)n^2) = O(k \cdot n^2)$.

3.3 Counting Colors

$O(1)$ for initializing the color counters. The graph G is a nested hash table that its length is the number of nodes n and the length of ”dictionary” at each index is the number of edges m .

1. the outer loop (‘for u in G do’): loop through the entire nodes in the graph so the outer iteration operates n times.
2. the inner loop (‘for v in G[u] do’): loop through edges connected to each node. The number of connected edges to nodes can be different. However, in the end, we have to loop for the number of edges, m , between the nodes in the graph.

Therefore, the running time of ‘ColorCounting(G)’ is $O(n + m)$.